

TRAYAN DAS

Lead AI Engineer · AI Architecture · Senior AI/ML Engineer · ~8 years (~6 in AI/ML)

trayandas@gmail.com · [+91-7001299506](tel:+91-7001299506) · linkedin.com/in/trayan-das · github.com/trayan4 · trayan4.github.io

PROFESSIONAL SUMMARY

AI/ML engineer and architect with ~8 years of professional experience, 6+ in AI/ML, designing and delivering production-grade generative AI, multi-agent, and machine learning platforms on Azure and GCP. Owns solution design end to end — multi-agent orchestration, multimodal and hybrid RAG over large enterprise document sets, and the LLMops around them: stage-wise evaluation harnesses, distributed tracing, cost governance, and CI/CD. Experienced in governance-first design — citation grounding enforced in code, guardrails, PII redaction, human-in-the-loop approval, identity-scoped access, and deciding which steps should not involve a model at all — and in translating business goals into measurable delivery outcomes.

TECHNICAL SKILLS

Languages: Python, SQL

AI Architecture & Governance: Solution architecture, multi-agent & RAG design patterns, AI guardrails, human-in-the-loop governance, tiered autonomy, PII redaction, cost/latency trade-off analysis

Generative AI & Agentic Systems: Multi-agent orchestration, Retrieval-Augmented Generation (RAG, hybrid BM25 + vector), prompt engineering, LLM fine-tuning (LoRA), Model Context Protocol (MCP), LangChain, LangGraph, CrewAI, Claude Agent SDK, Microsoft Foundry Agents, Copilot Studio Flows & Topics, Power Platform, Power Automate

Machine Learning & Deep Learning: PyTorch, scikit-learn, Hugging Face Transformers, LSTM autoencoders, Isolation Forest, Random Forest, Decision Tree, PCA, K-means, HDBSCAN, Linear & Logistic Regression, Ridge Regression, NLP

Observability & Evaluation: OpenTelemetry, LangSmith, Application Insights (KQL), MLflow, RAGAS, OpenEvals, LLM-as-Judge evaluation, golden-set eval harnesses, ROUGE

Cloud — Azure: AI Foundry, Azure OpenAI, Azure AI Search, Microsoft Fabric, Cosmos DB, Azure SQL, Databricks, Function App, App Service, Logic Apps, Container Apps, Storage

Cloud — GCP: BigQuery, Cloud Storage, App Engine

Data & Retrieval: Vector search — FAISS, Pinecone, Azure AI Search; Graph — Neo4j; Platforms — Microsoft Fabric, Azure SQL, Azure Databricks

Data Engineering: PySpark, ETL pipelines, feature engineering, data preprocessing, SQL

Tools & DevOps: Docker, Git, GitHub Actions CI/CD, Azure DevOps, Bicep IaC, MLOps & LLMops pipelines, CI/CD/CT, REST APIs, FastAPI, Flask

WORK EXPERIENCE

AI/ML Computational Science Specialist | Accenture

Sept 2026 – Present

- **Forensic Knowledge Assistant (Multimodal RAG Agent):** Designed and built a multimodal, citation-grounded Q&A assistant (Microsoft Foundry, Azure AI Search, Azure Functions, Copilot Studio, Power Platform) that answers engineering-investigation questions from a document knowledge base, accepting text, an image, or both from Teams and Copilot chat.
 - Implemented a Foundry reasoning agent (Claude) that answers from retrieved source chunks and the original user image. Under a cite-or-silent policy, it returns structured output: the answer, cited chunk IDs and a found flag. Paired it with a vision agent that turns user photos into fixed-template captions used to build the retrieval query.
 - Developed a Python Azure Function orchestrator (Foundry Responses API) that captions an optional image, queries Azure AI Search with the question and caption, and sends the image, question and retrieved chunks to the reasoning agent in one call. It handles text-only, image-only (with a default question) and mixed requests.
 - Enforced grounding in code rather than prompt alone: validated every cited chunk ID against the retrieved set, checked answer groundedness against the cited text, and returned a deterministic "not found" response when citations were missing or invalid.
 - Configured a Copilot Studio agent whose topic checks every message for attachments with a deterministic condition and calls a single agent flow, which invokes the Function through a Power Platform custom connector.
 - Solved the file hand-off from chat to backend by diagnosing that the flow converted attachments to raw bytes before the request was sent, then making the Function tolerant of single- and double-encoded base64 and validating image type before the model call.
 - Secured the solution with Entra ID: OAuth 2.0 on the custom connector and App Service Authentication on the Function, so every call carries the signed-in user's identity. A managed identity with least-privilege Azure roles handles outbound calls, so the same code runs locally and in Azure.
 - Established a stage-wise evaluation framework:
 - Captions: 90% pass under an SME-calibrated multimodal LLM judge, plus an oracle-caption retrieval ablation.
 - Retrieval: recall@10 of 0.84 on a golden dataset that includes image queries.
 - Answers: RAGAS factual correctness 0.87 and multimodal faithfulness, a custom per-citation precision judge, and a 2% false-answer / 5% false-refusal rate, all scored by a judge from a different model family than the one being judged.
 - System: deterministic routing tests, and p95 end-to-end latency of 8s text-only and 20s with an image.

- **Privacy-Safe Conversational Analytics Agent Pipeline:** Architected and led delivery of an end-to-end conversational analytics platform on Azure. It combined incremental ingestion with PII/profanity redaction, a hybrid BM25 + vector Search index over 4M+ source records consolidated into 100K+ indexed documents, and a four-agent LangGraph reasoning layer answering counting, ranking, trend and thematic questions. Cut time-to-insight from hours to seconds (~95% faster), retrieval latency by ~75% and analyst effort by 60–70%.
 - Developed a Python PII and profanity redaction pipeline using spaCy NER, regex pattern matching and the better-profanity library, applying SHA-256 hashing to identifiers so individuals could not be re-identified downstream.
 - Deployed ingestion as a timer-triggered Azure Function App with watermark-based incremental sync, detecting and copying only new telemetry records to avoid redundant reprocessing, and merging source records by conversation_id before chunking.
 - Fine-tuned and deployed two versions of GPT-4.1-nano multi-label classifiers for products & intent as Azure AI Search custom skills, enriching documents at indexing time rather than at query time.
 - Constructed a hybrid Search (BM25 + vector embeddings fused via Reciprocal Rank Fusion) over 100K+ conversation documents, using paginated lexical search and similarity-thresholded vector search with HNSW for low-latency retrieval.
 - Designed a LangGraph Router → Planner → Worker → Synthesizer system with four agents:
 - a router answering follow-ups from Cosmos DB-backed short-term memory;
 - a query-understanding agent producing a structured JSON plan (product, intent, time range, follow-up context);
 - a retrieval-and-analytics agent;
 - a synthesizer generating grounded, plain-English answers.
 - Engineered the analytics layer:
 - OData-filtered and faceted AI Search queries for counts and rankings;
 - parallel async searches per product × intent combination for multi product / intent queries;
 - chained Python tools (period-over-period, top-N, breakdowns, trends, share-of-total, spike detection) that read data from LangGraph state rather than LLM arguments.
 - Established component-level evaluation on frozen golden datasets:
 - routing accuracy 97% (200 questions) and query-parsing accuracy 81% (exact-match on all four plan fields, 300 questions);
 - retrieval set precision 0.91, set recall 0.92, and exact count match on 93% of queries, across 150 labelled queries;
 - answer quality 89% pass via LLM-as-judge over 200 questions, plus a deterministic check that every reported number matched source outputs.
 - Containerized the agent services with Docker and automated deployment through GitHub Actions CI/CD pipelines.
 - Instrumented end-to-end platform observability with OpenTelemetry and Application Insights, tuning the azure-monitor-opentelemetry-exporter and writing KQL queries to trace latency, token cost and failure modes across the agent chain.
- **Enterprise Data Q&A Agent (MCP-Grounded Text-to-SQL):** Designed and led development of an MCP-grounded text-to-SQL system (Microsoft Copilot Studio, LangChain on Azure Functions, Microsoft Foundry, Azure Container Apps) that translates natural-language questions into T-SQL executed against Microsoft Fabric Lakehouse SQL analytics endpoints, enabling analytics on 10M+ records in under 30 seconds.
 - Built Model Context Protocol (MCP) servers on Azure Container Apps, one per lakehouse dataset, exposing auto-generated table and column metadata enriched with SME-authored column descriptions as MCP resources, and query execution as MCP tools.
 - Architected a single query agent holding several dataset schemas in context at once, planning verified cross-dataset joins against SME-authored join keys and few-shot question-to-SQL examples held per dataset in the agent's prompt, with schema pulled per request.
 - Implemented catalog-driven routing: the agent reasons over short per-dataset descriptions to select which schemas to fetch, then pulls that grounding just-in-time, so a request carries only the schemas it needs.
 - Enforced per-user data access end to end via on-behalf-of token exchange — the caller's Microsoft Entra ID token is exchanged for a Fabric-scoped token so row-level security applies per user, with the servers' managed identity reserved for configuration and telemetry rather than data access.
 - Hardened query execution inside the server boundary with SELECT-only validation, row caps, query timeouts and an error-feedback self-correction loop, so guardrails hold regardless of the calling client.
 - Developed the query agent on LangChain (Azure Functions) as MCP client, after diagnosing Foundry IQ interference with schema grounding, behind a Copilot Studio orchestrator handling conversational intent, clarifying questions and Power Automate flows.
 - Engineered a Foundry output agent synthesizing single- and multi-dataset results into user-facing answers, gated by an LLM-as-a-judge evaluator (Azure Function) that scores each response against the SQL results before delivery.
 - Validated accuracy through SME review of generated T-SQL and results, plus a labelled 148-question test set measuring routing accuracy at 94%.
- **GenAI Business Document Generator:** Designed and delivered an agentic, multi-agent RAG system with LangGraph and a LoRA fine-tuned LLM for automated, context-aware business document generation on Azure — cutting manual drafting effort by 70–80% and enabling generation of 100+ documents per week.
 - Designed a LangGraph state-aware orchestration framework for managing multi-turn conversations across agents.
 - Developed a GenAI and core-Python pipeline to dynamically populate and update content within document templates.
 - Exposed conversation and document-generation endpoints through RESTful Flask APIs, integrating with Azure OpenAI for LLM inference.
 - Used Azure Cosmos DB for document metadata storage.
 - Instrumented the pipeline with LangSmith tracing and ROUGE-based evaluation to monitor generation quality across template types.

- **Well Performance Analytics Platform:** Implemented a PyTorch LSTM-autoencoder pipeline for anomaly detection on well-sensor time-series data, improving risk detection accuracy by ~20% and reducing analysis time by ~40%.
 - Performed extensive data preprocessing: handling missing values, outlier detection, normalization, and time-series alignment of multi-rate sensor signals.
 - Conducted feature engineering to extract domain-relevant indicators including ROP, WOB, torque trends, and vibration signatures.
 - Developed and evaluated LSTM-autoencoder models for anomaly detection and performance forecasting, tuning reconstruction-error thresholds against field-confirmed events.
- **Oil & Gas Insights Web Application:** Designed a PySpark KPI pipeline on Azure Databricks using PCA, K-means, and Random Forest to derive KPI features from equipment telemetry, reducing manual reporting overhead by ~50%.
 - Implemented preprocessing pipelines in PySpark to clean, join, and aggregate large-scale sensor datasets.
 - Engineered features from raw telemetry including rolling averages, lag features, and pressure differentials for downstream analytics.
 - Applied PCA for dimensionality reduction with K-means and Random Forest to segment and score equipment behaviour for KPI reporting.
 - Deployed interactive dashboards for field engineers to monitor live equipment metrics.

- **Equipment Failure Prediction Model:** Developed predictive maintenance models on GCP for oilfield equipment, achieving ~85% recall and reducing equipment downtime by ~30%.
 - Executed the end-to-end ML pipeline including data ingestion from GCP BigQuery, ETL processing, and feature engineering on time-series sensor data.
 - Trained and compared Isolation Forest (anomaly detection), Random Forest, Linear Regression, and Ridge Regression models for failure prediction.
 - Evaluated models using precision, recall, F1-score, and time-to-failure metrics; tuned thresholds for high-recall deployment targets.
 - Deployed trained models on GCP App Engine with scheduled inference jobs for continuous monitoring.
 - Maintained data pipelines and model versioning for reproducible retraining cycles.

- **Cross-functional Operations & Preventive Maintenance:** Led cross-functional teams responsible for medical equipment maintenance, biomedical engineering support and training, and inventory operations across client sites.

PROJECTS & PUBLICATIONS

RTI-Engine — AI Pay Transparency Compliance Engine [GitHub](#) | [Medium](#)

Architected an 11-node LangGraph multi-agent system for EU pay-transparency compliance — 6 model-driven nodes and 5 deliberately deterministic ones where no judgment is required.

- Designed the node split as an explicit architecture decision: classification, informational response, own-data response, regulatory positioning, drafting, and review are model-driven; statistics, figure validation, human approval, fallback, and not-applicable responses run as plain deterministic code.
- Implemented tiered autonomy enforced in code, with conditional human-approval gates that suspend the graph mid-run for sign-off before a response is released.
- Modelled a Neo4j knowledge graph parsed from the directive text — articles, jurisdictions, national provisions, and policy sections, with extracted article-to-article cross-references — and exposed it to the agents through MCP servers alongside hybrid vector retrieval.
- Added deterministic figure-grounding validation so every statistic in a draft is checked against source data rather than trusted from model output.
- Implemented cross-vendor model review, PII redaction, and model fallbacks for provider degradation.
- Deployed on Azure Container Apps with Bicep IaC, Docker and GitHub Actions CI/CD, and instrumented with LangSmith and OpenTelemetry tracing plus an openevals golden-set harness.

Autonomous Data Analyst [GitHub](#) | [Medium](#)

Engineered a LangGraph multi-agent text-to-SQL system on Azure with governance-first routing and an eval-first design — 87% accuracy on a 110-question golden-set harness.

- Implemented governance-first routing that classifies each request before execution and declines out-of-scope or unanswerable questions rather than generating speculative SQL.
- Implemented execution-repair validation loops that catch failing SQL, feed the database error back to the model, and retry — recovering the mechanical failures (hallucinated columns, wrong aliases, type mismatches) that dominate first-pass errors.
- Enforced root-cause decomposition so multi-part analytical questions are broken into sub-queries and answered with evidence rather than a single speculative query.
- Added model tiering to route simple steps to cheaper models and reserve the largest model for reasoning steps, with OpenTelemetry cost tracing to measure the saving.
- Established a golden-set eval harness spanning five assertion types — rubric-graded root-cause analysis, PII-guard blocking, numeric-tolerance comparison, categorical exact-match, and out-of-scope refusal.

Multimodal Agentic Reasoning Assistant (MARA) [GitHub](#) | [Medium](#)

- Designed a LangGraph multi-agent system coordinating RAG, Vision, Data Analysis, and Web Search agents for complex multi-modal queries, served over a production FastAPI REST API.
- Implemented parallel agent fan-out, dispatching 3–5 agents per query concurrently rather than sequentially, with end-to-end p95 latency ~30s dominated by LLM inference across the slowest branch.
 - Constructed hybrid retrieval fusing FAISS dense vectors with BM25 sparse search, reaching recall@10 ~0.75 on the evaluation query set.
 - Added cross-encoder reranking, accepting ~1–2s of additional overhead as a deliberate trade for retrieval quality on long documents.
 - Implemented adaptive planning with error recovery so a failing agent degrades the answer rather than the request.
 - Exposed the system through a FastAPI REST API with per-component timing instrumentation.

Hybrid RAG from Scratch [GitHub](#)

- Engineered a hybrid retrieval RAG pipeline from first principles, implementing the retrieval and fusion algorithms rather than calling a framework.
- Implemented BM25 sparse ranking from scratch, including term-frequency saturation and document-length normalization.
 - Implemented Reciprocal Rank Fusion from scratch to combine sparse and dense result lists without tuning a score-scale weighting.
 - Added FAISS dense retrieval with BGE embeddings and ran generation locally through Ollama, keeping the whole pipeline self-hosted.
 - Established a RAGAS-based LLM-as-judge evaluation harness to measure faithfulness and context quality across retrieval modes.

GPT-Style Language Model from Scratch [GitHub](#) | [Medium](#)

- Implemented a decoder-only 124M-parameter LLM from scratch in PyTorch, trained on 124M tokens, with an end-to-end generation pipeline.
- Implemented BPE tokenization and the full transformer block — multi-head causal self-attention, position embeddings, feed-forward layers, and residual connections — without high-level abstractions.
 - Wrote the training loop from scratch, including batching, learning-rate scheduling, gradient clipping, and validation-loss checkpointing.
 - Extended the pretrained model with instruction fine-tuning on a formatted instruction dataset.
 - Built an end-to-end text generation pipeline with temperature and top-k sampling controls.

Smart Compose System (Gmail-style) [GitHub](#)

- Developed a low-latency, Gmail-style smart compose system with systematic evaluation across fine-tuned and base models.
- Fine-tuned a GPT-2 model on email-style text for next-phrase suggestion in the composition flow.
 - Ran a systematic comparison of fine-tuned versus base models, measuring inference speed against contextual suggestion quality.
 - Optimized the inference path for interactive latency, since a suggestion that arrives after the user has typed the phrase has no value.

EDUCATION

M.Tech. (Master of Technology) University of Calcutta	2016 – 2018
B.Tech. (Bachelor of Technology) WBUT	2011 – 2015

CERTIFICATIONS

▪ AI-102: Microsoft Azure AI Engineer Associate	▪ Claude Certified Architect – Foundations — Anthropic
▪ AZ-900: Microsoft Azure Fundamentals	▪ CS50AI: Introduction to Artificial Intelligence with Python — Harvard University
▪ AI-900: Microsoft Azure AI Fundamentals	▪ LLMOps, MLOps & CI/CD/CT Pipelines — Udemy

TRAINING

▪ Microsoft Copilot Studio	▪ Deep Learning & Artificial Neural Networks
▪ Microsoft Fabric	▪ LLMOps, MLOps & CI/CD/CT Pipelines
▪ Model Context Protocol (MCP)	

AWARDS

Client Spotlight Award Accenture — delivery of the privacy-safe analytics agent platform	Feb 2026
Growth Catalyst Award Accenture — technical proficiency and project delivery	Dec 2025